

Zwischenbericht

Campus Next-Gen Data-Hub

Evaluation von Airbyte als ETL- und Integrationswerkzeug

Ablösung von Talend

INF-M Modul Projekte SoSe '26

Gruppe Airbyte

Lahres Timo

Bräutigam Rebecca

Horst Isabella

07.06.2026

Inhaltsverzeichnis

1	Projektziel & Kontext	2
2	Erreichte Meilensteine (Stand 06.06.2026)	2
2.1	Installation	2
2.2	Datenbasis (Source-PostgreSQL)	3
2.3	Angelegte Airbyte-Connectoren & erster ETL-Lauf	3
3	Architektur (Kurzüberblick)	4
4	Besonderheit: Datenqualität der Quell-CSVs	5
5	Probleme & Lösungen	6
5.1	Windows-/Umgebungsspezifische Hürden	6
5.2	Datenqualität der Quell-CSVs (Hauptproblem)	6
5.3	Airbyte-/abctl-spezifische Hürden	7
6	Offene Punkte / Fragen an die Betreuer	7
7	Anforderungs- und Szenarien-Status	8
8	Nächste Schritte	8
9	Anhang: Repository-Struktur	8

1 Projektziel & Kontext

Im Projekt evaluieren wir [Airbyte](#) als modernes ETL- und Datenintegrationswerkzeug mit dem Ziel, das bisher eingesetzte **Talend** in der Hochschul-IT abzulösen.

Die Evaluationsumgebung läuft **lokal in Docker Desktop** und stellt einen realistischen Ausschnitt der Hochschul-Datenlandschaft dar, einschließlich anonymisierter Studierenden-, Gebäude-, Instituts- und Personaldaten.

Die Evaluation ist in **sechs Testszenarien** gegliedert:

- DB-Connector
- Facility-Management-Sync
- Bild-BLOBs
- Studenten/Personal-Mapping
- Incremental Sync / IdM
- Web-APIs

Alle Szenarien sind detailliert in [testszenarien.md](#) beschrieben.

2 Erreichte Meilensteine (Stand 06.06.2026)

Tabelle 1: Meilenstein-Übersicht mit Bearbeitungsstand

Meilenstein	Status	Beleg / Doku
Installation des Systems	✓ DB-Stack + Airbyte lauffähig	installation-guide.md
Zugang für Betreuer	🟠 dokumentiert, Einrichtung im Termin	zugang.md
Einfacher ETL-Prozess	✓ durchgeführt und verifiziert (Postgres → Postgres)	etl-prozess.md
Beginn der Dokumentation	✓ README + 8 Dokumente unter docs/	dieses Repo

2.1 Installation

Das Setup ist vollständig skriptbasiert und reproduzierbar:

- `scripts/install.ps1` (Windows) bzw. `scripts/install.sh` (Linux/macOS) startet den kompletten Datenbank-Stack (Source-PostgreSQL, Ziel-PostgreSQL, Ziel-MySQL, CSV-File-Server), wartet auf den **healthy**-Status und lädt die Testdaten.

- `scripts/setup-airbyte.ps1` / `scripts/setup-airbyte.sh` installiert Airbyte Community Edition über das offizielle CLI `abctl` in einem lokalen Kubernetes-Cluster (`kind`) innerhalb von Docker Desktop.
- **Plattformen:** Das Setup steht für **Windows, Linux und macOS** bereit (identische Logik, plattformspezifische Skripte).

Alle vier Datenbank-/Server-Container laufen verifiziert im Zustand `healthy`.

2.2 Datenbasis (Source-PostgreSQL)

Die anonymisierten Testdaten sind geladen:

Tabelle 2: Tabellen in der Source-Datenbank und ihre Lademechanismen

Tabelle	Zeilen	Lademechanismus
fm_gebaeude	25	<code>scripts/load_fm_gebaeude.py</code>
fm_inst	2.083	<code>scripts/load_fm_inst.py</code>
k_plz	34.172	<code>scripts/load_k_plz.py</code>
fm_rna	379	<code>scripts/load_json.py</code>
hso_personal	870	<code>scripts/load_json.py</code>
hso_students	0	nur via File-Connector (s. Abschn. 2.3)
fm_stamm	0	keine Quelldatei (s. Abschn. 6)

2.3 Angelegte Airbyte-Connectoren & erster ETL-Lauf

In Airbyte sind folgende Connectoren angelegt und per Verbindungstest erfolgreich geprüft:

- **Sources (5):**
 - HSO Source PostgreSQL (Postgres, Update-Methode *User Defined Cursor*)
 - HSO CSV hso_students (local, /local/*.csv)
 - HSO CSV k_plz (local, /local/*.csv)
 - HSO CSV fm_gebaeude (local, /local/*.csv)
 - HSO CSV fm_inst (local, /local/*.csv)
- **- HSO Dest PostgreSQL (Port 5434)
 - HSO Dest MySQL (Port 3306, SSL aus, `allowPublicKeyRetrieval=true`, Raw-DB destdb)**

Es wurden **drei Connections** (jeweils *Full Refresh / Overwrite*) ausgeführt und das Ergebnis **direkt in der jeweiligen Ziel-DB** verifiziert:

Tabelle 3: Ausgeführte ETL-Connections und verifizierte Ergebnisse

Connection	Streams	Ergebnis (Ziel-DB)
HSO Source PostgreSQL → HSO Dest PostgreSQL	fm_gebaeude, k_plz	25 / 34.172 ✓
HSO Source PostgreSQL → HSO Dest MySQL	fm_gebaeude, k_plz	25 / 34.172 ✓
HSO CSV hso_students → HSO Dest PostgreSQL	hso_students (File)	5.052 Zeilen ✓

Bemerkenswert: Der File-Connector lud die strukturell defekte `hso_students.csv` vollständig (5.052 Zeilen) in die DB – dieselbe Datei, an der ein direktes PostgreSQL-COPY scheiterte (0 Zeilen, s. Abschn. 4). Pandas im File-Connector toleriert die Spalteninkonsistenzen. Damit sind **DB-Connector** (PG → PG, PG → MySQL) und **File-Connector** (CSV → DB) – der Kern von Szenario 1 inkl. „PostgreSQL → MySQL“ – nachgewiesen.

Zusätzlich stellt der Dienst **PostgREST** (`hso_postgrest`, Szenario 6a) eine REST-API auf die Ziel-DB bereit. Der Aufruf `GET http://localhost:3000/k_plz?limit=5` liefert die synchronisierten Daten als JSON.

3 Architektur (Kurzüberblick)

Die vollständige Beschreibung findet sich in [architektur.md](#). Alle Dienste laufen in Docker Desktop in einem gemeinsamen Netzwerk (`airbyte_net`); Airbyte selbst läuft in einem `kind`-Kubernetes-Cluster und erreicht die Datenbanken über `host.docker.internal`.

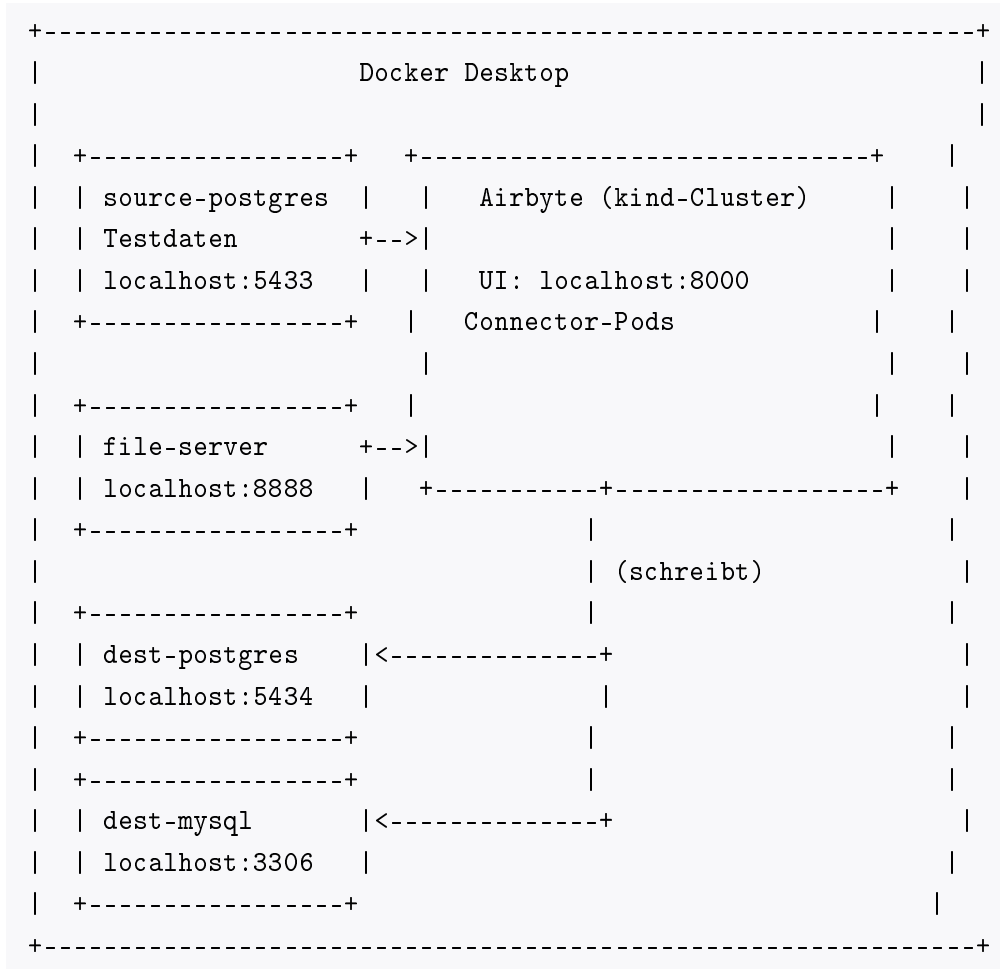


Abbildung 1: Systemarchitektur: Docker-Desktop-Netzwerk mit Airbyte im kind-Cluster

4 Besonderheit: Datenqualität der Quell-CSVs

Die bereitgestellten CSV-Dateien ließen sich **nicht** per direktem PostgreSQL-COPY laden (Details in Abschn. 5.2). Wir haben daher tolerante Python-Loader implementiert, die die Daten nach dem Containerstart bereinigt einspielen. Die Datei `hso_students.csv` ist strukturell so inkonsistent, dass sie aktuell nicht zuverlässig in die relationale Source-DB geladen werden kann; Studierendendaten werden stattdessen über den **Airbyte File-Connector** als Flatfile-Quelle eingebunden.

5 Probleme & Lösungen

5.1 Windows-/Umgebungsspezifische Hürden

Tabelle 4: Windows-spezifische Hürden und ihre Lösungen

Problem	Ursache	Lösung
<code>install.ps1</code> meldete „Python gefunden“, JSON-Laden schlug aber fehl	<code>python</code> ist unter Windows oft nur der Microsoft-Store-Platzhalter; <code>Get-Command</code> findet ihn, er liefert aber keine echte Version	Echte Versionsprüfung (<code>py/python/python3</code>); zusätzlich Docker-Fallback , der die Daten ganz ohne Host-Python lädt
<code>setup-airbyte.ps1</code> fand kein <code>abctl</code> -Asset	Falsches Namensschema – korrekt ist <code>abctl-<version>-windows-<arch>.zip</code> ; zudem liegt <code>abctl.exe</code> in einem Unterordner	Asset per Muster ermitteln, aus Unterordner entpacken; Architektur-Erkennung PowerShell-7-fest
File-Server-Container dauerhaft <code>unhealthy</code> , Setup lief in Timeout	Healthcheck nutzte <code>localhost</code> → im Container zuerst IPv6 (<code>:::1</code>), <code>nginx</code> lauscht aber nur auf IPv4	Healthcheck auf <code>127.0.0.1</code> umgestellt

5.2 Datenqualität der Quell-CSVs (Hauptproblem)

Der SQL-Init lud die drei Kern-Tabellen **gar nicht** – COPY brach unter `ON_ERROR_STOP` bereits an der ersten fehlerhaften Datei ab, wodurch alle folgenden leer blieben. Die konkreten Probleme je Datei:

- `fm_gebaeude.csv`: unquotierte Kommas in Textfeldern (z. B. „Hörsäle,Bib.,RZ“), eingebettete Header-Zeilen.
- `k_plz.csv`: **3.417 Header-Zeilen** mitten in den Daten (zusammengesetzte Einzel-Exporte), Zeilen mit zu wenig Feldern, ein Feld (`krskfz`) breiter als das Schema.
- `fm_inst.csv`: 86 Spalten (Tabelle nutzt 24), semikolon-getrennt, vereinzelt **NUL-Bytes**, doppelt kodierte UTF-8-Umlaute.
- `hso_students.csv`: **strukturell defekt** – Datenzeilen haben mehr Spalten (ca. 50–56) als der eigene Header (40), dazu unbalanciertes Quoting und Float-formatierte Integer.

Lösung: Tolerante Python-Loader (`scripts/load_*.py`) filtern Header, reparieren bzw. überspringen fehlerhafte Zeilen, bereinigen NUL-Bytes und Mojibake und führen Typkonvertierungen best-effort durch. Der SQL-COPY-Schritt wurde entfernt (`01_load_data.sql`).

5.3 Airbyte-/abctl-spezifische Hürden

Airbyte Community läuft über `abctl` in einem `kind`-Kubernetes-Cluster. Daraus ergaben sich mehrere nicht-triviale, in der offiziellen Dokumentation nicht beschriebene Hürden:

Tabelle 5: Airbyte-/abctl-spezifische Hürden und ihre Lösungen

Problem	Ursache	Lösung
File-Connector (<code>local</code>) findet <code>/local/*.csv</code> nicht	Connector-Pods (<code>kind</code>) sehen das Docker-Volume <code>oss_local_root</code> nicht	CSV-Verzeichnis beim Install via <code>abctl local install -volume "...:/local"</code> direkt in den Cluster mounten
<code>-volume</code> mit Windows-Pfad: <code>is not a valid volume spec</code>	<code>abctl</code> trennt den Volume-String stur an <code>:</code> , der Laufwerks-Doppelpunkt (<code>C:</code>) kollidiert	Pfad in MSYS-Form <code>/c/Users/...</code> angeben
Alle Connector-Tests/Syncs hängen Pending	Aktiviertes lokales Volume erwartet PVC <code>airbyte-local-pvc</code> in jedem Job-Pod; es existierte nicht (<code>persistentvolumeclaim not found</code>)	PV (<code>hostPath /local</code>) + PVC <code>airbyte-local-pvc</code> anlegen; <code>JOB_KUBE_LOCAL_VOLUME_ENABLED=true</code> + Neustart von <code>launcher/worker</code>
<code>abctl local credentials</code> schlägt fehl (<code>invalid character '<'</code>)	Kombinierter Aufruf löst einen Org-Lookup aus, der HTML statt JSON liefert	E-Mail und Passwort in zwei getrennten Aufrufen setzen (erst <code>-email</code> , dann <code>-password</code>)
Connector-Auswahl heißt „ Postgres “ (nicht „PostgreSQL“); Default-Update-Methode ist CDC	–	In der UI „Postgres“ wählen; Update-Methode auf <i>User Defined Cursor</i> stellen (CDC bräuchte <code>wal_level=logical</code>)

Alle Lösungen sind in `scripts/setup-airbyte.ps1/.sh` automatisiert und in [airbyte-setup.md](#) sowie [etl-prozess.md](#) dokumentiert.

6 Offene Punkte / Fragen an die Betreuer

- **hso_students.csv – Soll-Struktur:** Die Datei ist nicht eindeutig ladbar (mehr Datenspalten als Header-Einträge, defektes Quoting). Ist das Einlesen einer strukturell defekten Datei gewollt, oder wird eine korrekt lesbare Version bereitgestellt? Dies blockiert aktuell Szenario 4 (Account-Mapping in die Source-DB).

- **fm_stamm (Raumstammdaten):** Es liegt keine Quelldatei vor. Wird diese Tabelle benötigt, und woher sollen die Daten kommen?
- **Zugang für Betreuer:** Airbyte Community Edition ist im Wesentlichen Single-User und läuft auf `localhost`. Wie soll der Betreuer-Zugang erfolgen? Gemeinsamer Admin-Login während des Vor-Ort-Termins, oder ist ein Remote-Zugang (z.B. Tunnel/VM) gewünscht? (Vorschlag in [zugang.md](#).)
- **Sync-Strategie:** Wir nutzen den Cursor-Modus (`updated_at`) statt CDC, da CDC zusätzliche PostgreSQL-Konfiguration (`logical WAL`, `Replication Slot`) erfordert. Ist CDC für die Evaluation gewünscht?
- **Scope:** Welche der sechs Szenarien haben für die Bewertung Priorität?

7 Anforderungs- und Szenarien-Status

Eine vollständige Gegenüberstellung aller Kickoff-Anforderungen und der sechs Szenarien mit Bewertung der Airbyte-Eignung und unserem Umsetzungsstand findet sich in [anforderungen.md](#). Die wesentlichen Befunde:

- **Erfüllt:** Open Source/Community, PostgreSQL- & MySQL-Anbindung, CSV/JSON/Excel-Dateien, Logging/Monitoring, einfacher ETL-Lauf (verifiziert).
- **Einschränkungen gegenüber Talend:**
 - Kein OSS-Connector für **Informix**
 - Kein nativer **XML**-Connector
 - Keine freie Code-Snippet-Ausführung (Mapping nur via `dbt-SQL` oder Custom-Connector)

8 Nächste Schritte

- Szenario 1 abrunden: Sync nach MySQL + ein File → DB-Sync (Nachweis File-Connector-Last).
- Weitere Szenarien umsetzen: FM-Denormalisierung (`dbt/View`), BLOB-Bilder, Incremental Sync (`IdM`), Web-APIs (`PostgREST/SOAP`).
- Klärung der offenen Fragen aus Abschn. 6.

9 Anhang: Repository-Struktur

Siehe [README.md](#). Das vollständige Setup ist in unter 20 Minuten reproduzierbar:

- Windows: `scripts/install.ps1 + scripts/setup-airbyte.ps1`
- Linux/macOS: `scripts/install.sh + scripts/setup-airbyte.sh`